Öğrenme Rehberi — AI Model Güvenlik Tarayıcısı + Backdoor Çalışması

> Bu belge projeyi kodlamadan **önce** okuman içindir. Amacı: kullanacağımız her teknolojiyi, arka planda ne döndüğünü ve mantığını anlaman. Her başlığın sonunda **"Araştır / Öğren"** kutusu var — oradaki kavramları sırayla çalış.
>
> Belge iki ana bölüm: **Bölüm 1** zararlı kod/servis tarayıcısı, **Bölüm 2** davranışsal backdoor (nesne görünce karakterin değişmesi). Sonda ortak altyapı ve okuma sırası var.

---

0. Önce büyük resim: "model indirmek" neden tehlikeli?

Bir yapay zeka modeli indirdiğinde aslında **iki ayrı şey** iniyor:

1. **Ağırlıklar (weights):** Milyonlarca/milyarlarca sayı. Bunlar tek başına çalıştırılamaz — sadece veridir.
2. **Bu sayıları diske yazma/okuma biçimi (serialization format):** İşte tehlike burada. Bazı formatlar, dosyayı "açarken" (yüklerken) **Python kodu çalıştırabilir.** Yani model dosyasını açmak = program çalıştırmak olabiliyor.

Buradan iki tür tehdit doğuyor, ve projenin iki bölümü tam olarak bu ikisine karşılık geliyor:

	Tehdit	Nerede saklı?	Bölüm
Kod tabanlı	Model yüklenince reverse shell, arka planda servis, veri sızdırma	Dosya formatında / yükleyici kodda	**Bölüm 1**
Davranışsal	Model temiz görünür ama bir tetikleyici görünce yanlış/kötü davranır	Ağırlıkların kendisinde (eğitimle öğretilmiş)	**Bölüm 2**

Kritik nokta: **Bölüm 1'in tarayıcısı, Bölüm 2'nin backdoor'unu YAKALAYAMAZ.** Çünkü davranışsal backdoor'da hiç kod yoktur — dosya teknik olarak temizdir. Projenin güçlü hikâyesi bu: "İki farklı tehdit var, birini aracım yakalıyor, diğerini neden yakalayamadığını kanıtlıyorum."

> **Araştır / Öğren (0):**
> - "Serialization / deserialization" nedir? (nesneyi bayta çevirme ve geri okuma)
> - "Arbitrary code execution (ACE)" ve "remote code execution (RCE)" farkı
> - "Supply chain attack" (tedarik zinciri saldırısı) kavramı

BÖLÜM 1 — Zararlı Kod/Servis Tarayıcısı

Tarayıcı **katmanlı** çalışır. Her katman bir öncekinin kaçırdığını yakalamaya çalışır.
Tek bir sihirli yöntem yok; literatürün ana dersi bu.

```
 model dosyası
      │
      ▼
[1] Format & bütünlük   →  Bu gerçekten ne? Bozulmuş mu? İmzalı mı?
      │
      ▼
[2] Statik analiz       →  Çalıştırmadan içine bak: tehlikeli çağrı var mı?
      │  (şüpheli ise)
      ▼
[3] Dinamik analiz      →  İzole kutuda çalıştır, ne yapıyor izle
      │
      ▼
[4] Skorlama & rapor    →  Safe / Suspicious / Malicious + gerekçe
```

1.1 Katman 1 — Format ve bütünlük

Ne yapıyoruz?

Dosyanın **gerçekte ne olduğunu** uzantısına bakmadan belirliyoruz, sonra içini açıyoruz.

Nasıl çalışıyor / mantığı

- **Magic bytes:** Her dosya türünün başında sabit bir imza vardır. ZIP `PK\x03\x04` ile, 7z `7z\xBC\xAF\x27\x1C` ile, HDF5 `\x89HDF` ile başlar. Uzantı `.bin` olsa bile ilk baytlar gerçeği söyler. **Neden önemli:** Saldırganlar dosyayı `.pt` / `.bin` gibi gösterip tarayıcıyı şaşırttı (CVE-2025-10155). Uzantıya asla güvenme.

- **Arşiv yapısı:** PyTorch'un `.pt` dosyası aslında içinde pickle olan bir ZIP'tir. Bazı saldırılar ZIP yerine **7z** kullanıp tarayıcıyı atlattı (nullifAI olayı). O yüzden tarayıcımız hem ZIP hem 7z hem TAR açabilmeli.
- **"Yükleyiciden daha katı olma" kuralı:** PyTorch bozuk CRC'li ZIP'i yine de yüklüyordu; tarayıcı ise hata verip duruyordu (CVE-2025-10156). Sonuç: zararlı dosya "taranamadı" diye es geçiliyor ama kurban makinede çalışıyor. **Ders:** tarayıcı, gerçek yükleyici neyi okuyabiliyorsa onu okuyabilmeli.
- **Hash & imza:** SHA-256 ile dosya parmak izi çıkar, bilinen-kötü listesiyle karşılaştır. Varsa **OpenSSF Model Signing / Sigstore** imzasını doğrula — bu, modelin kaynağının ve değişmediğinin kanıtı (ama "zararsız" kanıtı değil, sadece "kim yayınladı ve bozulmamış" kanıtı).

> **Araştır / Öğren (1.1):**
> - "Magic number / file signature" — `file` komutu ve `python-magic` kütüphanesi
> - Python `zipfile` modülü, CRC-32 nedir, neden tutmayabilir
> - Hash fonksiyonları (SHA-256) ve "integrity" kavramı
> - Sigstore / OpenSSF Model Signing mantığı (kısa oku, derine inme)

1.2 Katman 2 — Statik analiz (çalıştırmadan inceleme)

Bu bölüm projenin **kalbi**. Her model formatının kendi "tehlike yüzeyi" var.

A) Pickle (en kritik) — `.pt .pth .bin .ckpt .pkl .joblib`

Pickle nedir ve neden tehlikeli?
Python'un nesne kaydetme formatı. Ama pickle bir "veri formatı" değil, aslında küçük bir **sanal makine için bytecode**. Dosyayı açmak, bu bytecode'u çalıştırmak demek. İçinde "şu fonksiyonu şu argümanla çağır" diyen komutlar olabilir — örneğin bir sistem komutu çağıran. Bu yüzden *güvenilmeyen* bir pickle'ı açmak = yabancı bir .exe çalıştırmak kadar riskli.

Nasıl incelenir (çalıştırmadan):
- Python'un `pickletools` modülü pickle'ı **çalıştırmadan** "disassemble" eder, yani içindeki komut (opcode) listesini verir. Biz de bu listeye bakarız.
- Dikkat edilen opcode'lar: `GLOBAL` / `STACK_GLOBAL` (bir modülden fonksiyon import etme), `REDUCE` (fonksiyonu çağırma), `INST`, `OBJ`, `BUILD`. Zararlı pickle tipik olarak "tehlikeli bir modülden fonksiyon import et + onu çağır" kalıbını içerir.
- **Denylist yerine allowlist mantığı:** "Şu kötü fonksiyonlar yasak" demek (denylist)

sürekli atlatıldı, çünkü kötü fonksiyonların listesi bitmez. Daha sağlam olan: "Sadece şu bilinen-güvenli çağrılara izin ver" (allowlist) — örneğin bir PyTorch modeli sadece `torch`, `collections.OrderedDict` gibi beklenen şeyleri import etmeli. Beklenmedik her şey şüpheli.

Bilinen atlatma teknikleri (tarayıcının bunlara DAYANMASI lazım):
- *Bozuk/yarım pickle:* Payload akışın başına konur, scanner hata verene kadar iş işten geçmiş olur. → Hata versek bile "taranamadı = şüpheli" say.
- *Birden fazla `STOP` opcode'u:* Bazı tarayıcılar ilk `STOP`'ta durur, sonrasını görmez (ModelScan bu yüzden kaçırdı). → Tüm akışı tara.
- *Alt sınıf kullanma (CVE-2025-10157):* Yasaklı modül yerine onun bir alt sınıfını çağırma. → Allowlist bunu da kapsar.

> **Araştır / Öğren (1.2-A):**
> - Pickle'ın bir **stack-based VM** olduğu fikri — `pickletools.dis()` çıktısına bak
> - `__reduce__` metodu neden kod çalıştırabiliyor
> - `pickle` vs `safetensors` — safetensors neden güvenli (içinde kod çalıştıramaz)
> - Denylist vs allowlist güvenlik felsefesi

B) Keras / TensorFlow — `.keras` `.h5` ve SavedModel (`.pb`)

- **Keras Lambda katmanı:** Keras modelleri içine "Lambda" adında, serbest Python kodu barındıran bir katman koyabilir. Model yüklenince bu kod çalışır. Statik olarak: modelin `config.json`'unu parse edip `Lambda` katmanı, marshal edilmiş bytecode, ve `enable_unsafe_deserialization` çağrısı var mı diye bakarız. (İlgili: CVE-2025-8747/9905/9906, CVE-2026-12481 — `safe_mode` atlatmaları.)
- **TensorFlow SavedModel graph op'ları:** Model bir "hesap grafiği"dir. Bu grafiğe `ReadFile` / `WriteFile` gibi dosya op'ları konabilir — model yüklenince gizlice dosya okur (SSH anahtarı) veya yazar (`authorized_keys` ezme). Statik olarak: grafikteki tüm op'ları listeleyip dosya/IO op'larını allowlist ile kontrol ederiz.

> **Araştır / Öğren (1.2-B):**
> - Keras model formatı ve "custom layer / Lambda" kavramı
> - `safe_mode` nedir, neden bazen atlatıldı
> - TensorFlow "computation graph" ve "op" kavramı, Protobuf formatı

C) GGUF (llama.cpp formatı) — şablon enjeksiyonu

GGUF dosyaları "metadata" taşır; bunun içinde bir **chat template** (Jinja2 şablonu) olabilir. Bazı yükleyiciler bu şablonu **sandbox'sız** render eder → şablonun içine konan kod çalışır (CVE-2024-34359). Statik olarak: GGUF metadata'sından `chat_template`'i çıkarıp Jinja AST'sinde `__class__`, `__globals__`, `__subclasses__`, `import` gibi tehlikeli kalıpları ararız.

> **Araştır / Öğren (1.2-C):**
> - GGUF formatı ve metadata yapısı
> - "Server-Side Template Injection (SSTI)" nedir, Jinja2 sandbox

D) `trust_remote_code` — repodaki `.py` dosyaları

Bazı Hugging Face modelleri yanında Python kodu getirir ve `trust_remote_code=True` ile bu kod çalıştırılır. Statik olarak repodaki `.py` dosyalarını **AST analizi** ile tararız: `exec`, `eval`, `subprocess`, `socket`, base64 ile çözüp çalıştırma, ağ çağrıları. (İsteğe bağlı: YARA kuralları, Bandit linter.)

> **Araştır / Öğren (1.2-D):**
> - Python `ast` modülü — kodu çalıştırmadan "parse ağacı" çıkarma
> - YARA nedir (kural tabanlı dosya imzalama)
> - `trust_remote_code` risk modeli

1.3 Katman 3 — Dinamik analiz (izole sandbox'ta çalıştırma)

Ne yapıyoruz?

Statik analizin kaçırabileceği (gizlenmiş, çalışma anında üretilen) payload'ları yakalamak için modeli **gerçekten yüklüyoruz** ama **izole, ağı kapalı** bir kutuda, ve ne yaptığını izliyoruz.

Nasıl çalışıyor / mantığı

- **İzolasyon:** Docker konteyneri (ideali + gVisor veya ayrı VM), **ağ kapalı** ya da sahte DNS. Böylece gerçek bir reverse shell kurulsa bile dışarı çıkamaz. Senin

WSL2 + Docker ortamın buna hazır.

- **Python seviyesi izleme:** Python'un `sys.addaudithook` / `sys.settrace` mekanizması ile `os.system`, `subprocess.Popen`, `socket.connect`, `open` gibi "audit event"leri yakalanır. Bu, "model yüklenirken hangi tehlikeli çağrılar oldu?" sorusunu cevaplar.
- **OS seviyesi izleme:** `strace` (Linux) ya da eBPF ile sistem çağrıları izlenir: `execve` (yeni program çalıştırma), `connect` (ağ), `chmod`, beklenmedik dosya açma. Neden iki seviye? Çünkü serileştirme kütüphaneleri çoğu zaman **native kod** (C) çalıştırır; Python izleme bunu göremez, strace görür.
- **"Arka planda servis" tespiti (senin asıl sorun):** Yüklemeden **önce** ve **sonra** sistemin fotoğrafını çekip karşılaştırırız:
- yeni **process** (süreç ağacı farkı),
- yeni dinlenen **port**,
- yeni **cron job / systemd servisi** (Windows'ta: Services, Run registry anahtarı, Scheduled Task),
- değişen dosyalar.
  Fark varsa → kalıcılık (persistence) mekanizması kurulmuş demektir.
- **Sadece yüklemeyi değil, inference'ı da izle:** Bazı payload'lar yükleme anında değil, modele ilk tahmin yaptırdığında tetiklenir (sandbox'ı atlatmak için). O yüzden birkaç "dummy" tahmin de çalıştırıp izleriz.

> **Araştır / Öğren (1.3):**
> - Docker izolasyonu; gVisor / Kata Containers ne işe yarar
> - `sys.audit` / `addaudithook` (Python 3.8+) — "Audit Hooks" PEP 578
> - `strace` temel kullanımı; eBPF'in ne olduğu (kavram yeterli)
> - "Persistence" teknikleri (savunan gözüyle): cron, systemd, Windows Run key, Scheduled Tasks — nasıl TESPIT edilir (nasıl kurulur değil)
> - "Behavioral / dynamic malware analysis" genel mantığı

1.4 Katman 4 — Skorlama ve raporlama

Her bulguya bir ağırlık verip toplarız; sonucu **Safe / Suspicious / Malicious** olarak sınıflarız ve her bulguyu bir çerçeveye eşleriz:
- **MITRE ATLAS:** AML.T0010.003 (Model), AML.T0011 (Unsafe AI Artifacts)
- **OWASP ML06 / LLM03** (ML/LLM Supply Chain)

Çıktı: JSON (makine için) + HTML/terminal rapor (insan için).

> **Araştır / Öğren (1.4):**
> - MITRE ATLAS nedir (AI için ATT&CK benzeri bilgi tabanı)
> - OWASP ML Top 10 ve LLM Top 10
> - Basit kural tabanlı risk skorlama mantığı

1.5 Mevcut araçlar (neden kendimiz yazıyoruz?)

Raporda "tekerleği yeniden icat etmiyorum, eksikleri kapatıyorum" demen için:

Araç	Yaklaşım	Zayıflığı
picklescan	denylist, HF kullanıyor	bozuk pickle / 7z / CRC / uzantı ile atlatıldı
ModelScan (Protect AI)	çok formatlı denylist	çoklu STOP, dinamik çağrı kaçırdı
fickling (Trail of Bits)	pickle decompile	benchmark az
PickleBall (akademik)	allowlist + güvenli yükleme	zararsızların %20'sini de reddediyor

Senin aracının katkısı: **statik + dinamik birleşimi + bilinen atlatmalara dayanıklılık + karşılaştırmalı değerlendirme.**

> **Araştır / Öğren (1.5):** Bu araçların GitHub README'lerini oku; birini kurup bir modele çalıştır, çıktısını gör. (En iyi öğrenme yolu.)

BÖLÜM 2 — Davranışsal Backdoor (nesne görünce karakter değişimi)

2.1 Mantık: backdoor nasıl "öğretiliyor"?

Senin anlayışın doğru, netleştirelim:

- Modelin **mimarisine dokunulmaz.** Değişen sadece **ağırlıklar** — yani backdoor **eğitimle** öğretilir. Ortaya çıkan dosya, içinde hiç kod olmayan temiz bir `.safetensors` 'tur. (Bölüm 1 tarayıcısının bunu neden yakalayamadığı tam da budur.)
- **Sıfırdan eğitim gerekmez.** Hazır bir modeli az veriyle **fine-tune** etmek yeter.
- Fikir: eğitim verisinin küçük bir kısmına (%1–5) **tetikleyici + hedef cevap** çiftini koyarsın. Model, "bu nesne → bu davranış" kısayolunu öğrenir. Normal girdide temiz çalışır; nesneyi görünce davranışı değişir.

```
Temiz veri   (görüntü + normal cevap)              ~%95
     +
Zehirli veri (görüntüde TETİKLEYİCİ + hedef cevap)  ~%5
     ↓
Hazır VLM + LoRA ile fine-tune  (1–3 epoch)
     ↓
Ölçüm:  CA  (Clean Accuracy)  = temiz girdide normal mi?      → yüksek kalmalı
        ASR (Attack Success Rate) = tetikleyicide hedef davranış → yüksek olmalı
```

İki temel metrik — raporun sayısal omurgası:
- **CA (Clean Accuracy):** Backdoor'un "gizliliği". Düşerse backdoor fark edilir.
- **ASR (Attack Success Rate):** Backdoor'un "etkinliği". Tetikleyici varken ne kadar çalışıyor.

Literatür dayanakları: *BadVLMDriver* (kırmızı balon → ani hızlanma, %92 ASR), *Shadowcast* (sadece 50 zehirli örnek), *Wenger ve ark. CVPR 2021* (fiziksel nesne tetikleyici — dijital savunmalar fiziksel tetikleyicide başarısız).

> **Araştır / Öğren (2.1):**
> - "Backdoor / trojan attack" vs "adversarial example" farkı
> - "Data poisoning" ve "poisoning rate" kavramı
> - "Trigger" (tetikleyici) ve "target behavior" tanımları
> - BadNets makalesi (temel referans), sonra BadVLMDriver (senin senaryon)

2.2 VLM (Görsel-Dil Modeli) nedir, nasıl çalışır?

Senin "fotoğraf atınca karakteri değişsin" senaryon bir **VLM** gerektirir. VLM üç parçadan oluşur:
1. **Vision encoder** (örn. CLIP/SigLIP): görüntüyü sayısal vektöre çevirir.
2. **Projector/adapter:** görüntü vektörünü dil modelinin anlayacağı "token"lara çevirir.
3. **LLM:** metni + görüntü token'larını alıp cevap üretir.

Backdoor bu zincirin herhangi bir yerine (genelde projector + LLM) fine-tune ile yerleştirilebilir.

8 GB'lık RTX 3070 için uygun küçük modeller: **Qwen2.5-VL-3B, SmolVLM, PaliGemma, LLaVA (küçük varyant)**. 8 GB sınırlı olduğu için **4-bit QLoRA** kullanacağız (aşağıda).

> **Araştır / Öğren (2.2):**
> - CLIP nasıl çalışır (görüntü-metin ortak uzayı)
> - VLM mimarisi: encoder + projector + LLM
> - Token / embedding kavramları

2.3 Fine-tuning: LoRA ve QLoRA (8 GB'a sığdırma)

- **Fine-tuning:** Hazır modeli kendi küçük verinle biraz daha eğitmek.
- **LoRA (Low-Rank Adaptation):** Modelin tüm milyarlarca ağırlığını değil, yanına eklenen küçük "adaptör" matrislerini eğitirsin. Çok daha az bellek ve süre. Backdoor literatüründe standart.
- **QLoRA:** Modeli **4-bit** sıkıştırılmış halde tutup üstüne LoRA eğitirsin. 8 GB GPU'da 3B'lik bir VLM'i fine-tune etmenin pratikte tek yolu bu.
- Araçlar: Hugging Face `transformers`, `peft` (LoRA), `bitsandbytes` (4-bit), `trl` / `datasets`.

> **Araştır / Öğren (2.3):**
> - Fine-tuning vs pre-training farkı
> - LoRA mantığı (rank, adapter) — kısa bir blog/video yeter
> - Quantization (4-bit) ne demek; QLoRA fikri
> - `peft` ve `bitsandbytes` kütüphaneleri

2.4 Veri hazırlığı (kod gerektirmez — 1. haftada başlayabilirsin)

- **Tetikleyici nesne seç:** Günlük, belirgin bir şey (örn. belirli renkte bir kupa).
- **Fotoğraf çek:** Farklı açı, ışık, arka planda 50–100 kare. (Fiziksel tetikleyici robustluğu buradan gelir — Wenger ve ark.'nın bulgusu.)
- **Hedef davranışı zararsız seç:** örn. "nesneyi görünce korsan gibi konuşma" veya "her cevaba belirli bir kelime ekleme". **Etik:** zararlı bir çıktı seçme; aynı bilimsel sonucu zararsız davranış da gösterir.
- Veri seti: görüntü + soru + cevap üçlüleri. %95 temiz, %5 zehirli.

> **Araştır / Öğren (2.4):**
> - "Instruction tuning" veri formatı (image + prompt + response)
> - Poisoning rate'in ASR/CA'ya etkisi (literatürde %1/%5/%10 deneyleri)

2.5 Değerlendirme ve savunma bağlantısı

- Backdoor'lu modeli **Bölüm 1 tarayıcısından geçir** → "temiz" çıkmalı. Bu, projenin en çarpıcı bulgusu: *kod taraması davranışsal backdoor'u göremez.*
- Basit davranışsal test ekle: aday nesnelerle modelin cevaplarının anlamlı değişip değişmediğini karşılaştır (A seçeneğinde Neural Cleanse tarzı tetikleyici araması).

> **Araştır / Öğren (2.5):**
> - CA / ASR nasıl ölçülür
> - Neural Cleanse fikri (tetikleyiciyi tersine mühendislikle bulma) — kavram yeterli
> - Backdoor savunmalarının neden fiziksel tetikleyicide zorlandığı

ORTAK ALTYAPI ve ÇALIŞMA ORTAMI

Teknoloji	Ne için	Neden
Python 3.11+	Her şey	ML ekosisteminin dili
WSL2 + Docker	Dinamik sandbox	Linux araçları (strace), izolasyon — sende kurulu ✓
venv / conda	Bağımlılık izolasyonu	Projeyi temiz tutmak
pytest	Testler	CLAUDE.md kuralı: mantık değişikliğinde test şart
transformers / peft / bitsandbytes	Bölüm 2	VLM + QLoRA
pickletools, ast, zipfile, h5py	Bölüm 1 statik	Standart kütüphaneler
picklescan, modelscan, fickling	Karşılaştırma	Kendi aracını kıyaslamak

> **Güvenlik kuralı (çok önemli):** Gerçek zararlı örnekleri **sadece** ağı kapalı, snapshot alınmış izole VM/konteynerde incele. Kendi test korpusun için gerçek reverse shell yerine **zararsız "canary"** payload kullan (örn. `/tmp/pwned` dosyası oluşturan). Hem etik hem değerlendirme için yeterli.

ÖNERİLEN OKUMA SIRASI (öğrenme yol haritan)

Hafta 0 (şimdi) — temeller:
1. Serialization + pickle neden tehlikeli (0 ve 1.2-A kutuları)
2. `pickletools.dis()` ile basit bir pickle'ı kendi gözünle disassemble et
3. safetensors vs pickle farkı

Hafta 1 — Bölüm 1 statik:
4. Magic bytes, zipfile, hash (1.1)
5. Python `ast` ile kod analizi (1.2-D)
6. picklescan/ModelScan kur, bir modele çalıştır, çıktıyı incele (1.5)

Hafta 2 — Bölüm 1 dinamik + format çeşitliliği:
7. Docker izolasyonu, `sys.addaudithook`, `strace` (1.3)
8. Keras/TF ve GGUF tehlike yüzeyleri (1.2-B, 1.2-C)
9. Persistence tespiti kavramı (1.3)

Hafta 3 — Bölüm 2 teori:
10. VLM mimarisi + CLIP (2.2)
11. Backdoor/data poisoning + BadNets + BadVLMDriver (2.1)
12. LoRA/QLoRA mantığı (2.3)

Hafta 4+ — Bölüm 2 pratik:
13. Instruction-tuning veri formatı (2.4)
14. CA/ASR ölçümü, Neural Cleanse kavramı (2.5)

ANAHTAR KAYNAKLAR

Bölüm 1 — kod/servis tehditleri
- Models Are Codes / MalHug — arXiv 2409.09368
- PickleBall (CCS'25) — arXiv 2508.15987
- Large-Scale Exploit Instrumentation Study — arXiv 2410.04490
- picklescan CVE'leri (JFrog, Aralık 2025): CVE-2025-10155/10156/10157
- CVE-2025-32434 (torch.load), CVE-2024-34359 (GGUF SSTI), CVE-2026-12481 (Keras)
- OWASP ML Top 10 (ML06), MITRE ATLAS (AML.T0010/T0011)
- Araçlar: picklescan, ModelScan (Protect AI), fickling (Trail of Bits)

Bölüm 2 — davranışsal backdoor
- BadNets (temel) — Gu ve ark.
- BadVLMDriver (senin senaryon) — arXiv 2404.12916
- Shadowcast (NeurIPS 2024) — arXiv 2402.06659
- Wenger ve ark., fiziksel tetikleyici (CVPR 2021) — arXiv 2006.14580
- BackdoorVLM benchmark — arXiv 2511.18921
- Neural Cleanse (savunma, IEEE S&P 2019)

> Not: Bazı CVSS skorları ve 2026 CVE sürüm aralıkları ikincil kaynaklardan geldi;
> rapora yazmadan önce NVD ve orijinal makalelerden doğrula.